

第一次上机作业：YOLOv5

陈天乐 无 25
ctl22@mails.tsinghua.edu.cn

2025 年 6 月 4 日

目录

1	背景	2
2	作业任务	2
2.1	使用 YOLOv5s 完成推理，计算检测性能	2
2.2	可视化预测的结果，并分析典型失败案例	2
2.3	调整可能调整的配置	4
2.4	提出改进建议	7
2.4.1	主干网络（Backbone）优化	7
2.4.2	注意力机制增强	7
2.4.3	锚框匹配策略优化	7
2.4.4	轻量化与速度优化	7
3	源代码	8

1 背景

YOLOv5 is the world's most loved vision AI.本次上机作业要求使用官方提供的 YOLOv5 COCO 预训练权重（不再训练模型）和已经给出的 367 张测试图片和对应的标注（COCO 目标类别）完成推理任务。

2 作业任务

2.1 使用 YOLOv5s 完成推理，计算检测性能

将官网上的 yolov5 模型下载至本地后，编写了一个 coco_test.yaml 文件，用于定义测试验证的图片数据集，同时由于 annotations.json 中的 id 和 category 都是不连续的，因此编写了一个 Label.py 用于读出 annotations.json 中的 id 和对应的 category，写入 coco_test.yaml 中，代码在报告结尾给出。在终端调用 val.py 程序对给出的测试图片进行验证：

```
python val.py --weight yolov5s.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
```

得到 mAP@50 和 mAP@50:5:95 的结果如下图所示：

```
PS C:\Users\skyhappy\Desktop\学校的破事\作业\数字图像处理\DIPI_assignment\yolov5-master\yolov5-master> python val.py --weight yolov5s.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
val: data=./data/coco_test.yaml, weights=['yolov5s.pt'], batch_size=1, imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs\val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0+cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing layers...
YOLOv5s summary: 213 layers, 7225885 parameters, 0 gradients, 16.4 GFL0Ps
val: Scanning C:\Users\skyhappy\Desktop\image_path... 367 images, 233 backgrounds, 0 corrupt: 100%|██████████| 367/367
val: WARNING Cache directory C:\Users\skyhappy\Desktop is not writeable: [WinError 183] : 'C:\Users\skyhappy\Desktop\image_path.cache.npy' -> 'C:\Users\skyhappy\Desktop\image_path.cache'

```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	367	599	0.531	0.472	0.379	0.272

```
Speed: 0.2ms pre-process, 4.3ms inference, 0.9ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\val\exp18
```

图 1: 使用 YOLOv5s 完成推理的检测性能

2.2 可视化预测的结果，并分析典型失败案例

在终端调用 detect.py 对给出的图片数据集进行推理，得到带有预测框的输出图片，并将 label 保存到一个 txt 文件中：

```
python detect.py --source ImageSet --weights yolov5s.pt --img 640 --save-txt
```

然后编写了一个 Draw_GT.py 文件，在带有预测框的输出图片中将 annotations.json 的 GT 框画上，代码在报告结尾给出。这里给出几个典型失败案例和其分析原因（白色框为 GT 框，其余颜色是推理框）：



图 2: 失败案例 1

模型对这个图片发生了虚警，可能是由于标注的 class 中没有老虎这一类别，因此模型将该图片检测为了 bear。

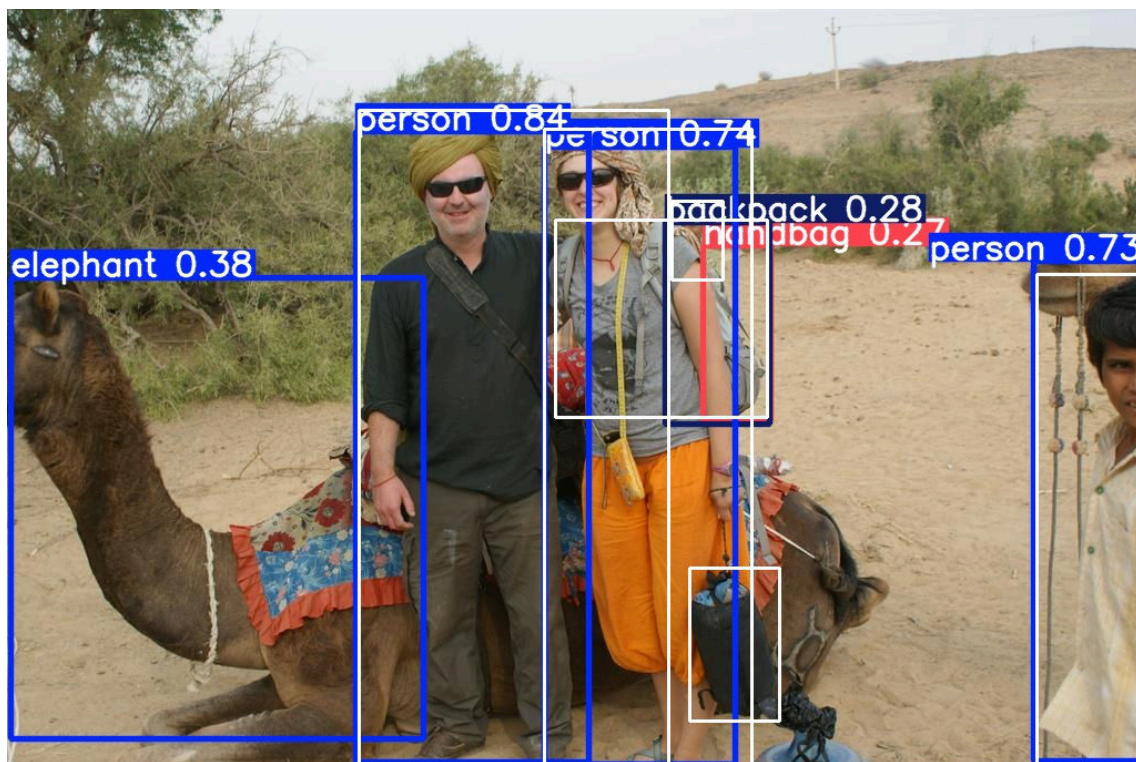


图 3: 失败案例 2

在这个图片中，模型正确识别了图中的人，但是在中间这个女人身上还有衣服和背包两种

物品未能识别出来，这是由于 YOLO 的算法导致的，对于很多重叠在一起的物体时，YOLO 只能对一整个物体进行识别，而不能对这个物体重叠部分的小物体进行识别。

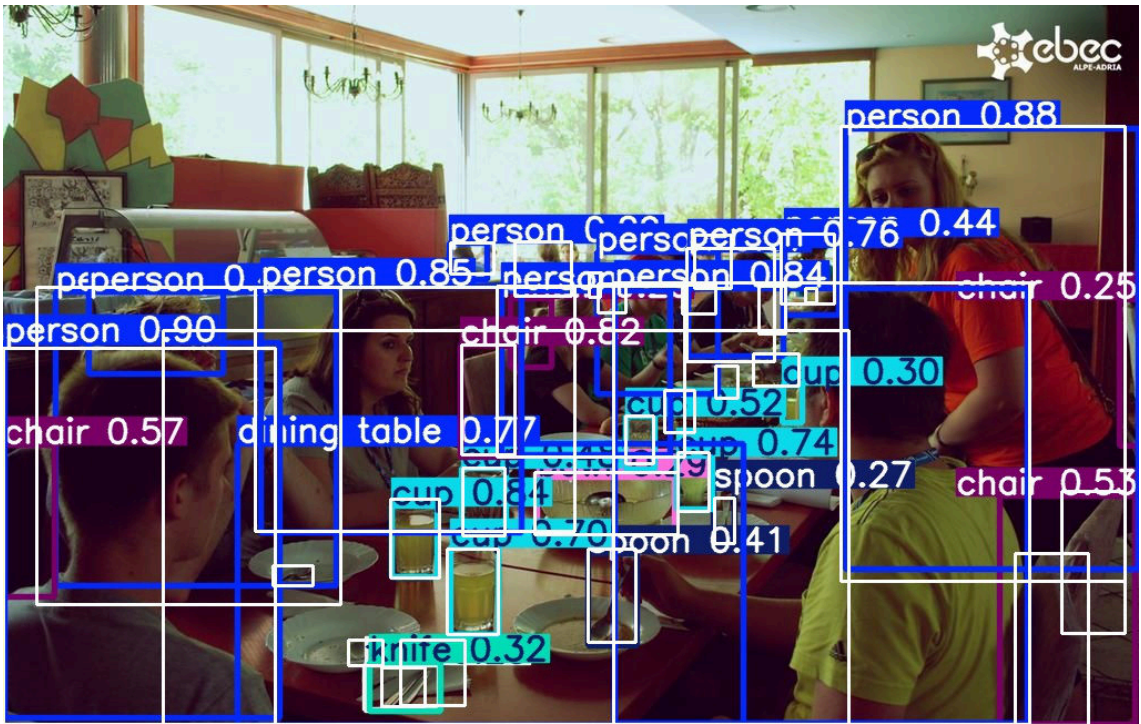


图 4: 失败案例 3

在这张图片中，物体分布的非常紧凑，同时还有一些非常细小的物体被标注为 GT。该模型基本能够把大部分物体识别出来，但是对于一些非常小型、细节的物体未能识别，这同样也是由于 YOLO 算法的原因。

2.3 调整可能调整的配置

对这批图片使用了不带 TTA 的模型 (n/m/l/x/s6) 和带 TTA 的模型 (n/m/l/x)，运行结果如下图：

```
PS C:\Users\skyhappy\Desktop\学校的破事\作业\数字图像处理\DIp_assignment\yolov5-master\yolov5-master> python val.py --weights yolov5n.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
val: data=./data/coco_test.yaml, weights=['yolov5n.pt'], batch_size=1, imgsz=640, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs\val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0+cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing layers...
YOLOv5n summary: 213 layers, 1867405 parameters, 0 gradients, 4.5 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\DIp_HW\image... 334 images, 33 backgrounds, 0 corrupt: 100%|██████████| 367/367
val: WARNING: Cache directory C:\Users\skyhappy\Desktop\DIp_HW is not writeable: [WinError 183]: 'C:\Users\skyhappy\Desktop\DIp_HW\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\DIp_HW\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100%|██████████| 367/367 [00:
all 367 984 0.648 0.503 0.591 0.447
Speed: 0.2ms pre-process, 4.2ms inference, 1.1ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\val\exp4
```

图 5: YOLOv5n 模型的检测性能

```
P5 C:\Users\skyhappy\Desktop\学校的课程\作业\数字图像处理\VID_assignment\yolov5-master\yolov5-master python val.py --weight yolov5m.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
val: data=./data/coco_test.yaml, weights=[yolov5m.pt], batch_size=1, imgsz=640, conf_thres=0.801, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save
e_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA-0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5m summary: 299 layers, 21172173 parameters, 0 gradients, 48.9 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 0% | 0/367 [00:00<7, 71it/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dictionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dictionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dictionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: WARNING Cache directory C:\Users\skyhappy\Desktop\VID_IP_HM is not writable: [WinError 183]: 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:
all 367 988 0.678 0.68 0.786 0.633
Speed: 0.2ms pre-process, 6.6ms inference, 0.9ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\val\exp5
```

图 6: YOLOv5m 模型的检测性能

```
P5 C:\Users\skyhappy\Desktop\学校的课程\作业\数字图像处理\VID_assignment\yolov5-master\yolov5-master python val.py --weight yolov5l.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
val: data=./data/coco_test.yaml, weights=[yolov5l.pt], batch_size=1, imgsz=640, conf_thres=0.801, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save
e_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA-0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5l summary: 367 layers, 46533693 parameters, 0 gradients, 189.0 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 0% | 0/367 [00:00<7, 71it/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dictionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dictionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: WARNING Cache directory C:\Users\skyhappy\Desktop\VID_IP_HM is not writable: [WinError 183]: 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:07:00:00, 56.55it/s]
all 367 988 0.796 0.678 0.67 0.552
Speed: 0.1ms pre-process, 9.9ms inference, 0.9ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\val\exp6
```

图 7: YOLOv5l 模型的检测性能

```
P5 C:\Users\skyhappy\Desktop\学校的课程\作业\数字图像处理\VID_assignment\yolov5-master\yolov5-master python val.py --weight yolov5x.pt --data ./data/coco_test.yaml --task val --img 640 --batch-size 1
val: data=./data/coco_test.yaml, weights=[yolov5x.pt], batch_size=1, imgsz=640, conf_thres=0.801, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save
e_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA-0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Downloading https://github.com/ultralytics/yolov5/releases/download/v7.8/yolov5x.pt to yolov5x.pt...
100% | 166M/166M [00:03:00:00, 55.7MB/s]

Fusing Layers...
YOLOv5x summary: 444 layers, 86705005 parameters, 0 gradients, 205.5 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 0% | 0/367 [00:00<7, 71it/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 56.46it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\VID_IP_HM is not writable: [WinError 183]: 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:09:00:00, 39.09it/s]
all 367 988 0.629 0.61 0.661 0.538
Speed: 0.2ms pre-process, 16.1ms inference, 0.9ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\val\exp7
```

图 8: YOLOv5x 模型的检测性能

```
P5 C:\Users\skyhappy\Desktop\学校的课程\作业\数字图像处理\VID_assignment\yolov5-master\yolov5-master python val.py --weight yolov5s6.pt --data ./data/coco_test.yaml --task val --img 1280 --batch-size 1
val: data=./data/coco_test.yaml, weights=[yolov5s6.pt], batch_size=1, imgsz=1280, conf_thres=0.801, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=False, verbose=False, save
e_txt=False, save_hybrid=False, save_conf=False, save_json=False, project=runs/val, name=exp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA-0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Downloading https://github.com/ultralytics/yolov5/releases/download/v7.8/yolov5s6.pt to yolov5s6.pt...
100% | 24.0M/24.0M [00:01:00:00, 15.9MB/s]

Fusing Layers...
YOLOv5s6 summary: 289 layers, 12612588 parameters, 0 gradients, 16.8 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 0% | 0/367 [00:00<7, 71it/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: Scanning C:\Users\skyhappy\Desktop\VID_IP_HM\image...: 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 56.46it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\VID_IP_HM is not writable: [WinError 183]: 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\VID_IP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:11:00:00, 31.49it/s]
all 367 988 0.601 0.605 0.696 0.583
Speed: 0.6ms pre-process, 12.2ms inference, 1.7ms NMS per image at shape (1, 3, 1280, 1280)
Results saved to runs\val\exp8
```

图 9: YOLOv5s6 模型的检测性能

```
PS C:\Users\skyhappy\Desktop\学校的故事\作业\数字图像处理\YOIP_assignment\yolov5-master\yolov5-master> python val.py --weight yolov5n.pt --data ./data/coco_test.yaml --task val --img 832 --augment --batch-size 1
val: datar./data/coco_test.yaml, weights=[yolov5n.pt], batch_size=1, imgsz=832, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=True, verbose=False, save
_txt=False, save_hybrid=False, save_conf=False, save_json=False, projectruns=val, nameexp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5n summary: 213 layers, 1867405 parameters, 0 gradients, 4.5 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 0% | 0/367 [00:00<7, 71t/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 56.54it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\YOIP_HM is not writable: [WinError 133] : 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:08:00:00, 41.48it/s]
all 367 984 0.526 0.582 0.632 0.496
Speed: 0.3ms pre-process, 12.7ms inference, 1.6ms NMS per image at shape (1, 3, 832, 832)
Results saved to runs\val\expl0
```

图 10: 带 TTA 的 YOLOv5n 模型的检测性能

```
PS C:\Users\skyhappy\Desktop\学校的故事\作业\数字图像处理\YOIP_assignment\yolov5-master\yolov5-master> python val.py --weight yolov5m.pt --data ./data/coco_test.yaml --task val --img 832 --augment --batch-size 1
val: datar./data/coco_test.yaml, weights=[yolov5m.pt], batch_size=1, imgsz=832, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=True, verbose=False, save
_txt=False, save_hybrid=False, save_conf=False, save_json=False, projectruns=val, nameexp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5m summary: 290 layers, 21172173 parameters, 0 gradients, 48.9 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 55.15it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\YOIP_HM is not writable: [WinError 133] : 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:11:00:00, 31.17it/s]
all 367 984 0.674 0.666 0.739 0.618
Speed: 0.3ms pre-process, 20.6ms inference, 1.2ms NMS per image at shape (1, 3, 832, 832)
Results saved to runs\val\expl1
```

图 11: 带 TTA 的 YOLOv5m 模型的检测性能

```
PS C:\Users\skyhappy\Desktop\学校的故事\作业\数字图像处理\YOIP_assignment\yolov5-master\yolov5-master> python val.py --weight yolov5l.pt --data ./data/coco_test.yaml --task val --img 832 --augment --batch-size 1
val: datar./data/coco_test.yaml, weights=[yolov5l.pt], batch_size=1, imgsz=832, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=True, verbose=False, save
_txt=False, save_hybrid=False, save_conf=False, save_json=False, projectruns=val, nameexp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5l summary: 367 layers, 46533693 parameters, 0 gradients, 109.0 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 57.89it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\YOIP_HM is not writable: [WinError 133] : 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:16:00:00, 22.39it/s]
all 367 984 0.64 0.634 0.664 0.539
Speed: 0.2ms pre-process, 33.0ms inference, 1.1ms NMS per image at shape (1, 3, 832, 832)
Results saved to runs\val\expl2
```

图 12: 带 TTA 的 YOLOv5l 模型的检测性能

```
PS C:\Users\skyhappy\Desktop\学校的故事\作业\数字图像处理\YOIP_assignment\yolov5-master\yolov5-master> python val.py --weight yolov5x.pt --data ./data/coco_test.yaml --task val --img 832 --augment --batch-size 1
val: datar./data/coco_test.yaml, weights=[yolov5x.pt], batch_size=1, imgsz=832, conf_thres=0.001, iou_thres=0.6, max_det=300, task=val, device=, workers=8, single_cls=False, augment=True, verbose=False, save
_txt=False, save_hybrid=False, save_conf=False, save_json=False, projectruns=val, nameexp, exist_ok=False, half=False, dnn=False
YOLOv5 2025-4-4 Python-3.9.17 torch-2.6.0-cu118 CUDA:0 (NVIDIA GeForce RTX 4060 Ti, 8188MiB)

Fusing Layers...
YOLOv5x summary: 480 layers, 86705005 parameters, 0 gradients, 205.5 GFLOPs
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 0% | 0/367 [00:00<7, 71t/s]Error decoding JSON from C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json. Starting with an empty dict
ionary.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\skyhappy\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dirpath/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
val: Scanning C:\Users\skyhappy\Desktop\YOIP_HM\image... 334 images, 33 backgrounds, 0 corrupt: 100% | 367/367 [00:06:00:00, 56.11it/s]
val: WARNING Cache directory C:\Users\skyhappy\Desktop\YOIP_HM is not writable: [WinError 133] : 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache.npy' -> 'C:\Users\skyhappy\Desktop\YOIP_HM\image.cache'
Class Images Instances P R mAP50 mAP50-95: 100% | 367/367 [00:25:00:00, 14.16it/s]
all 367 984 0.597 0.605 0.657 0.529
Speed: 0.2ms pre-process, 89.0ms inference, 1.1ms NMS per image at shape (1, 3, 832, 832)
Results saved to runs\val\expl3
```

图 13: 带 TTA 的 YOLOv5x 模型的检测性能

将 mAP@50 和 mAP@50:5:95 的数据汇总为下表:

表 1: 不带 TTA 的模型检测精度

模型	mAP@50	mAP@50:5:95
YOLOv5s	0.379	0.272
YOLOv5n	0.287	0.200
YOLOv5m	0.406	0.316
YOLOv5l	0.442	0.358
YOLOv5x	0.462	0.374

表 2: 带 TTA 的模型检测精度

模型	mAP@50	mAP@50:5:95
YOLOv5s	0.384	0.297
YOLOv5x	0.423	0.341

观察统计数据后可以发现，更换不同模型能够显著影响检测精度，因为不同模型的参数量大小不同，但对于该图片测试集而言，并不是参数量最大的 x 模型表现最好，反而是 m 模型表现较好，这可能是模型过拟合导致的。同时，带有 TTA 的模型只有 n 型有较明显的提高，剩余的甚至会有下降，但是基本不变。调整模型的 `conf_thres` 后，对模型的检测精度影响也不是非常明显。

2.4 提出改进建议

根据 yolo 之后的版本以及 yolov5 出现的问题，我认为其可以从以下几个部分提高：

2.4.1 主干网络（Backbone）优化

参考 YOLOv8 的改进，其引入了轻量化视觉变换器（RepViT），结合了 CNN 的局部感知能力和 ViT 的全局建模能力，能在减少参数量的同时提升检测精度，尤其适合处理复杂背景和小目标场景，这很大程度上能缓解 YOLOv5 对小物体难以识别的困难。同时，参考 YOLOv6 多尺度特征融合的特点，主干网络可以采用跨阶段局部网络（CSPNet）优化特征提取效率，结合快速空间金字塔池化（SPPF）模块，既保留多尺度信息，又降低计算复杂度。

2.4.2 注意力机制增强

参考 YOLOv8 的改进，在主干网络的关键位置插入轻量级 Transformer 块，结合局部卷积与全局自注意力机制，可以提升复杂场景下的特征表达能力，这样遇到复杂背景图片时检测性能可以大大提高。同时，Transformer 的一大特点就是注意力机制，在 Backbone 中加入位置坐标注意力，通过动态调整特征权重，增强关键区域的响应，同样可以提升微小目标的检测精度。

2.4.3 锚框匹配策略优化

参考 YOLOv7 的动态正样本选择策略改进，可以优化 Anchors 的匹配策略，调整锚框尺寸以适应不同数据集，例如为微小目标设计更密集的锚框，采用动态匹配规则，增加正样本数量以加快收敛速率。

2.4.4 轻量化与速度优化

观察到 YOLOv5 不同网络的推理速度在 CPU 上运行还需要几百毫秒，对于实时应用而言延时可能还是会有点大。因此可以对网络进行轻量化处理，比如进行量化、剪枝操作，还可以在量化时采用动态量化技术（如 INT8 量化），在保证精度的前提下压缩模型体积，提升推理速度，适用于边端部署和实时系统。

3 源代码

```
1 import json
2 import time
3 import os
4 from tqdm import tqdm
5 from ultralytics import YOLO
6
7
8 # COCO 官方80类的原始ID (非连续列表)
9 COCO_OFFICIAL_IDS = [
10     1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 13, 14, 15, 16, 17, 18, 19, 20,
11     21, 22, 23, 24, 25, 27, 28, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
12     41, 42, 43, 44, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58,
13     59, 60, 61, 62, 63, 64, 65, 67, 70, 72, 73, 74, 75, 76, 77, 78, 79,
14     80, 81, 82, 84, 85, 86, 87, 88, 89, 90
15 ]
16
17 # 配置参数 (根据实际情况修改)
18 annotations_json_path = "C:/Users/skyhappy/Desktop/学校的破事/作业/数字图像处理/DIP_assignment
    /annotations.json" # COCO标注文件路径
19 output_path = "C:/Users/skyhappy/Desktop/DIP_HW" # 输出目录
20 image_path = "C:/Users/skyhappy/Desktop/DIP_HW/images" # 图片目录
21 val_path = os.path.join(output_path, 'image.txt')
22
23
24 # 创建映射字典: {原始COCO_ID : 连续YOLO_ID (0-79)}
25 COCO_TO_YOLO_ID = {coco_id: yolo_id for yolo_id, coco_id in enumerate(COCO_OFFICIAL_IDS)}
26
27 def coco2yolo(annotations_json_path, output_path, image_path):
28     # 创建输出目录
29     labels_dir = os.path.join(output_path, 'Label')
30     os.makedirs(labels_dir, exist_ok=True)
31
32
33 # 加载COCO标注
34 with open(annotations_json_path, 'r') as f:
35     coco_data = json.load(f)
36     image_entries = coco_data.get("images", [])
37
38     images_info = {img['id']: img for img in coco_data['images']}
39
40     # 按图像分组标注
41     annotations = {}
42     for ann in coco_data['annotations']:
43         img_id = ann['image_id']
44         if img_id not in annotations:
45             annotations[img_id] = []
46         annotations[img_id].append(ann)
47
48     for img_id, img_info in tqdm(images_info.items(), desc="Processing Images"):
49         # 获取图片信息
50         img_name = img_info['file_name']
51         img_width = img_info['width']
52         img_height = img_info['height']
53
54         # 验证图片存在
55         img_path = os.path.join(image_path, img_name)
56         if not os.path.exists(img_path):
57             print(f"Warning: Image {img_name} not found, skipping")
58             continue
```

```

59
60     with open(val_path, 'w') as f:
61         for entry in image_entries:
62             name = entry.get("file_name")
63             f.write(f"C:/Users/skyhappy/Desktop/DIP_HW/images/{name}\n")
64     f.close
65
66     # 创建对应的标签文件
67     label_name = os.path.splitext(img_name)[0] + '.txt'
68     label_path = os.path.join(labels_dir, label_name)
69
70     # 写入YOLO格式标注
71     with open(label_path, 'w') as f:
72         if img_id in annotations:
73             for ann in annotations[img_id]:
74                 # 检查是否为COCO 80类
75                 coco_id = ann['category_id']
76                 if coco_id not in COCO_TO_YOLO_ID:
77                     continue # 跳过非COCO80类的标注
78
79                 # 获取连续YOLO ID
80                 class_id = COCO_TO_YOLO_ID[coco_id]
81
82                 # 转换边界框格式,归一化
83                 x, y, w, h = ann['bbox']
84                 x_center = (x + w / 2) / img_width
85                 y_center = (y + h / 2) / img_height
86                 w_norm = w / img_width
87                 h_norm = h / img_height
88
89                 # 写入: class_id x_center y_center width height
90                 f.write(f"{class_id} {x_center:.6f} {y_center:.6f} {w_norm:.6f} {h_norm:.6f}\n")
91
92 if __name__ == "__main__":
93     # 执行转换
94     coco2yolo(annotations_json_path, output_path, image_path)
95     print(f"Conversion complete! Labels saved to: {os.path.join(output_path, 'Label')}")

```

Listing 1: Label.py

```

1 import json
2 import os
3 import shutil
4 from pathlib import Path
5 import numpy as np
6 import cv2
7 from tqdm import tqdm
8 import argparse
9 import yaml
10
11
12 def parse_args():
13     parser = argparse.ArgumentParser(description='YOLOv5检测结果可视化工具')
14     parser.add_argument('--img-dir', type=str, default='C:/Users/skyhappy/Desktop/DIP_HW/exp3',
15                         help='预测输出图片目录')
16     parser.add_argument('--label-dir', type=str, default='C:/Users/skyhappy/Desktop/DIP_HW/Label',
17                         help='真实标签目录')
18     parser.add_argument('--output-dir', type=str, default='C:/Users/skyhappy/Desktop/DIP_HW/visual_results',
19                         help='可视化结果输出目录')
20     parser.add_argument('--yaml-path', type=str, default='C:/Users/skyhappy/Desktop/DIP_HW/coco_test.yaml',
21                         help='数据集配置文件路径')

```

```

18     parser.add_argument('--gt-color', type=tuple, default=(255, 255, 255), help='真实框颜色 (
19         BGR) ')
20
21     return parser.parse_args()
22
23 def load_class_names(yaml_path):
24     """从YAML文件加载类别名称"""
25     with open(yaml_path, 'r', encoding='utf-8') as f:
26         data = yaml.safe_load(f)
27         return data['names']
28
29 def draw_boxes(image, label_path, color, class_names, is_gt=True):
30     h, w = image.shape[:2]
31     with open(label_path, 'r', encoding='utf-8') as f:
32         for line in f:
33             parts = line.strip().split()
34             if not parts:
35                 continue
36
37             try:
38                 if is_gt:
39                     cls_id, xc, yc, bw, bh = map(float, parts)
40                     conf = None
41             except Exception as e:
42                 print(f"错误: 标签文件 {label_path} 中存在格式错误的行: {line.strip()}, 错误信
43                     息: {e}")
44                 continue
45
46             # 坐标转换
47             x1 = int((xc - bw/2) * w)
48             y1 = int((yc - bh/2) * h)
49             x2 = int((xc + bw/2) * w)
50             y2 = int((yc + bh/2) * h)
51             # 绘制边界框
52             cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
53
54             if not is_gt:
55                 cls_id = int(cls_id)
56                 if cls_id >= len(class_names):
57                     print(f"警告: 预测类别ID {cls_id} 超出类别列表范围 (共{len(class_names)}
58                         类)")
59                     continue
60                 label = f"{class_names[cls_id]} {conf:.2f}"
61                 (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
62                 cv2.rectangle(image, (x1, y1-20), (x1+tw, y1), color, -1)
63                 cv2.putText(image, label, (x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
64                     (255,255,255), 2)
65
66     return image
67
68 def main(args):
69     os.makedirs(args.output_dir, exist_ok=True)
70     class_names = load_class_names(args.yaml_path)
71     processed = 0
72     for img_name in os.listdir(args.img_dir):
73         if not img_name.lower().endswith(('.jpg', '.jpeg', '.png')):
74             continue
75
76         base_name = os.path.splitext(img_name)[0]
77         img_path = os.path.join(args.img_dir, img_name)

```

```

76     gt_label_path = os.path.join(args.label_dir, f"{base_name}.txt")
77     pred_label_path = os.path.join(args.pred_dir, f"{base_name}.txt")
78
79     print(f"\n处理图片: {img_name}")
80     print(f"真实标签路径: {gt_label_path}")
81     print(f"预测标签路径: {pred_label_path}")
82
83     image = cv2.imread(img_path)
84     if image is None:
85         print(f"错误: 无法读取图片 {img_path}, 请检查文件格式或路径。")
86         continue
87
88     image = draw_boxes(image, gt_label_path, args.gt_color, class_names, is_gt=True)
89
90     output_path = os.path.join(args.output_dir, img_name)
91     cv2.imwrite(output_path, image)
92     processed += 1
93     print(f"成功保存: {output_path}")
94
95
96 if __name__ == "__main__":
97     args = parse_args()
98     main(args)

```

Listing 2: Draw_GT.py

```

1  # Ultralytics  AGPL-3.0 License - https://ultralytics.com/license
2  # parent
3  #     yolov5
4  #     datasets
5  #         coco_test
6
7  # Train/val/test sets as 1) dir: path/to/imgs, 2) file: path/to/imgs.txt, or 3) list: [path/to
8  #     /imgs1, path/to/imgs2, ...]
9  path: C:/Users/skyhappy/Desktop/DIP_HW # dataset root dir
10 train: image.txt # train images (relative to 'path')
11 val: image.txt # val images (relative to 'path')
12 test: image.txt # test images (optional)
13 labels_dir: Label
14
15 # Classes
16 names:
17     0: person
18     1: bicycle
19     2: car
20     3: motorcycle
21     4: airplane
22     5: bus
23     6: train
24     7: truck
25     8: boat
26     9: traffic light
27     10: fire hydrant
28     11: stop sign
29     12: parking meter
30     13: bench
31     14: bird
32     15: cat
33     16: dog
34     17: horse
35     18: sheep
36     19: cow

```

```
36 20: elephant
37 21: bear
38 22: zebra
39 23: giraffe
40 24: backpack
41 25: umbrella
42 26: handbag
43 27: tie
44 28: suitcase
45 29: frisbee
46 30: skis
47 31: snowboard
48 32: sports ball
49 33: kite
50 34: baseball bat
51 35: baseball glove
52 36: skateboard
53 37: surfboard
54 38: tennis racket
55 39: bottle
56 40: wine glass
57 41: cup
58 42: fork
59 43: knife
60 44: spoon
61 45: bowl
62 46: banana
63 47: apple
64 48: sandwich
65 49: orange
66 50: broccoli
67 51: carrot
68 52: hot dog
69 53: pizza
70 54: donut
71 55: cake
72 56: chair
73 57: couch
74 58: potted plant
75 59: bed
76 60: dining table
77 61: toilet
78 62: tv
79 63: laptop
80 64: mouse
81 65: remote
82 66: keyboard
83 67: cell phone
84 68: microwave
85 69: oven
86 70: toaster
87 71: sink
88 72: refrigerator
89 73: book
90 74: clock
91 75: vase
92 76: scissors
93 77: teddy bear
94 78: hair drier
95 79: toothbrush
```

Listing 3: coco_test.yaml